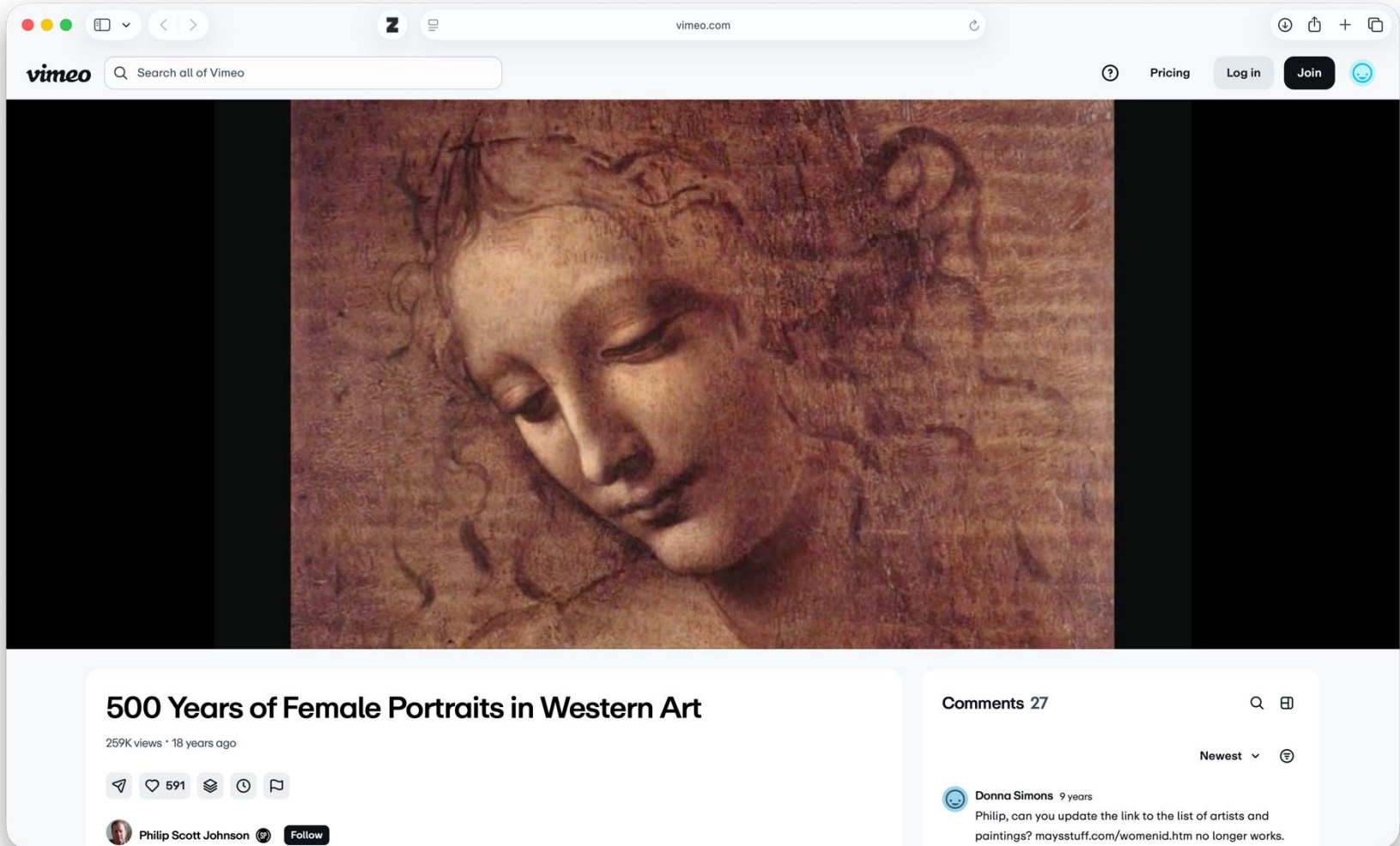


# Amuse-bouche



<https://vimeo.com/1456037>

# Image Warping and Morphing

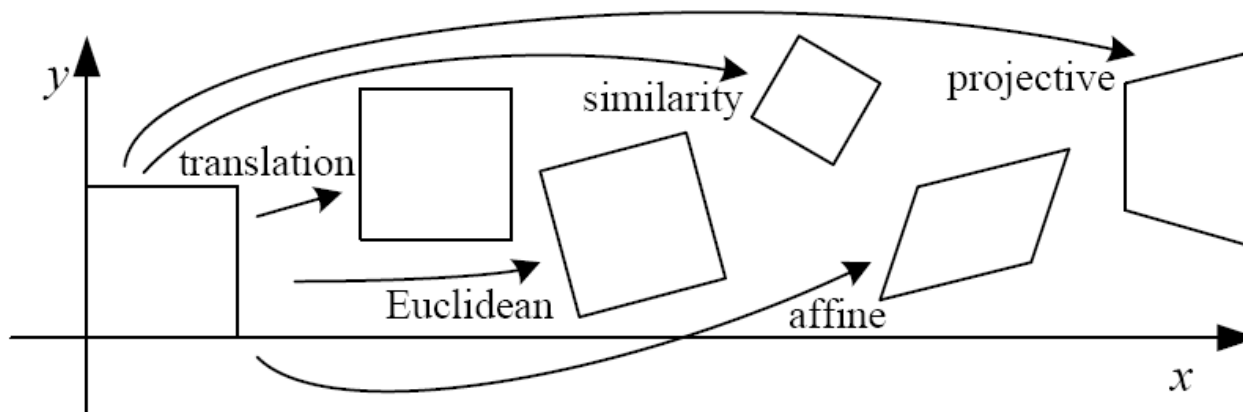
---



© Alexey Tikhonov

CS180/280A: Intro to Computer Vision and Comp. Photo  
Alexei Efros and Ren Ng, UC Berkeley, Fall 2026

# 2D image transformations



| Name              | Matrix                                                                | # D.O.F. | Preserves: | Icon |
|-------------------|-----------------------------------------------------------------------|----------|------------|------|
| translation       | $\begin{bmatrix} \mathbf{I} & \mathbf{t} \end{bmatrix}_{2 \times 3}$  |          |            |      |
| rigid (Euclidean) | $\begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix}_{2 \times 3}$  |          |            |      |
| similarity        | $\begin{bmatrix} s\mathbf{R} & \mathbf{t} \end{bmatrix}_{2 \times 3}$ |          |            |      |
| affine            | $\begin{bmatrix} \mathbf{A} \end{bmatrix}_{2 \times 3}$               |          |            |      |
| projective        | $\begin{bmatrix} \tilde{\mathbf{H}} \end{bmatrix}_{3 \times 3}$       |          |            |      |

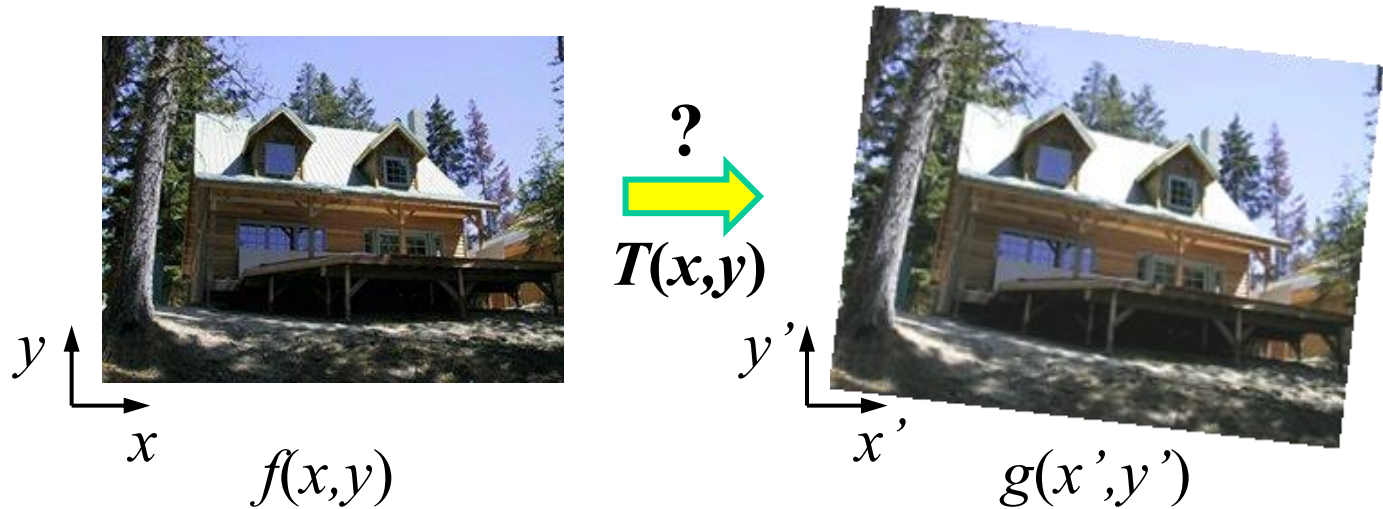
These transformations are a nested set of groups

- Closed under composition and inverse is a member



# Recovering Transformations

---

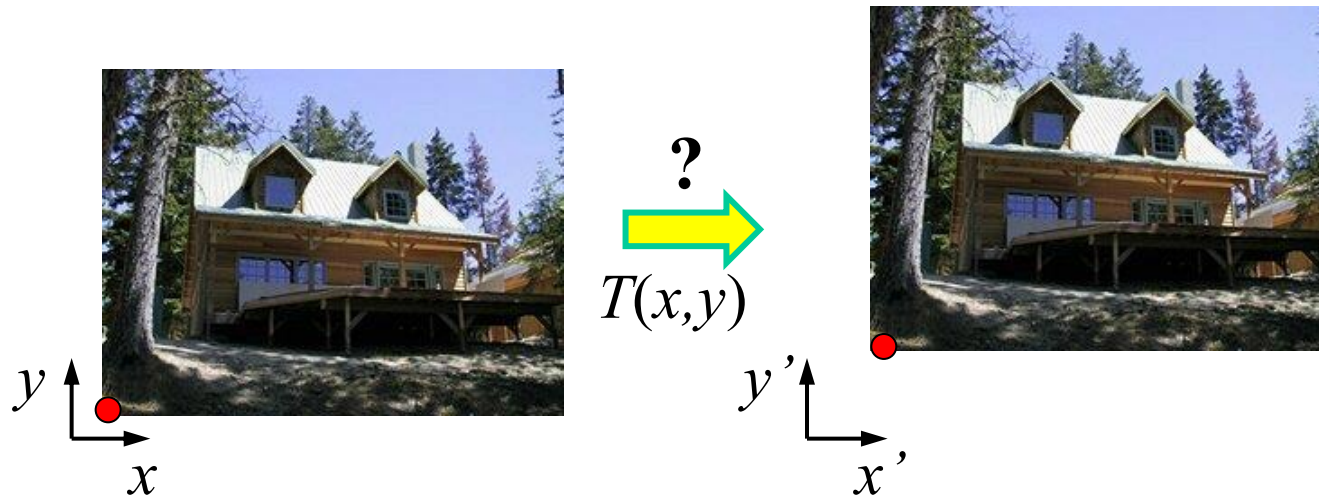


What if we know  $f$  and  $g$  and want to recover the transform  $T$ ?

- e.g. better align images from Project 1
- willing to let user provide correspondences
  - How many do we need?

# Translation: # correspondences?

---



How many correspondences needed for translation?

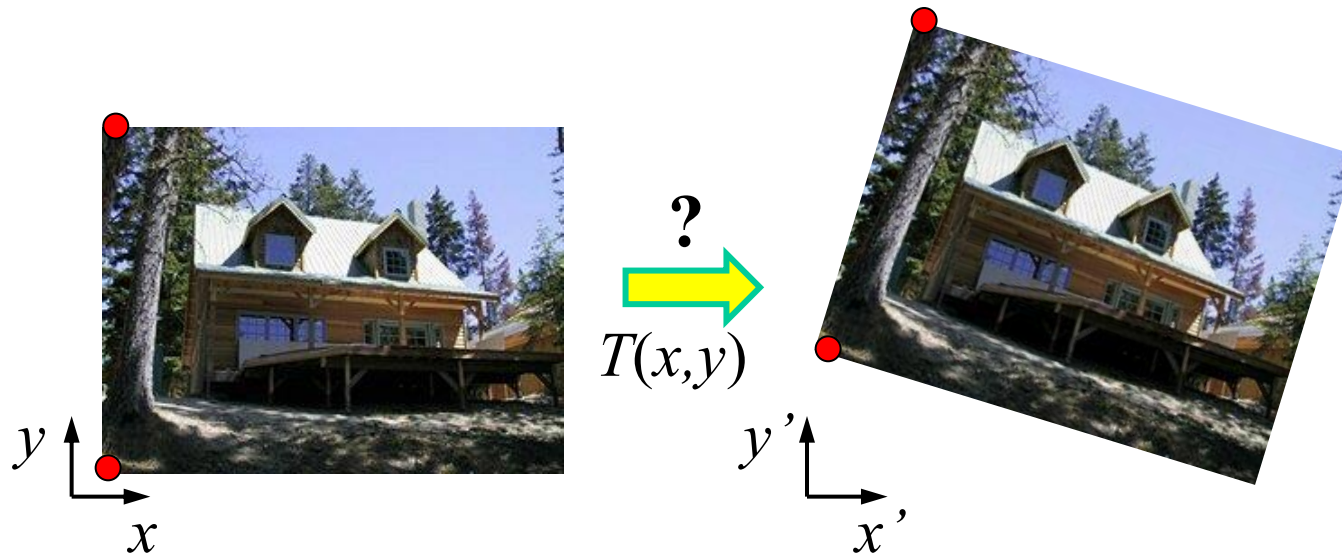
How many Degrees of Freedom?

What is the transformation matrix?

$$\mathbf{M} = \begin{bmatrix} 1 & 0 & p'_x - p_x \\ 0 & 1 & p'_y - p_y \\ 0 & 0 & 1 \end{bmatrix}$$

# Euclidian: # correspondences?

---

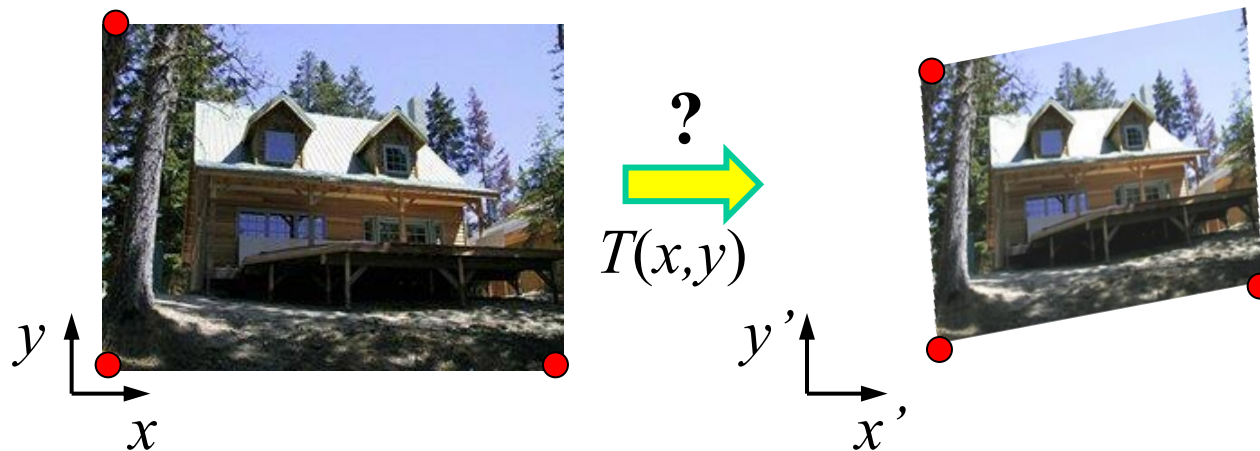


How many correspondences needed for translation+rotation?

How many DOF?

# Affine: # correspondences?

---

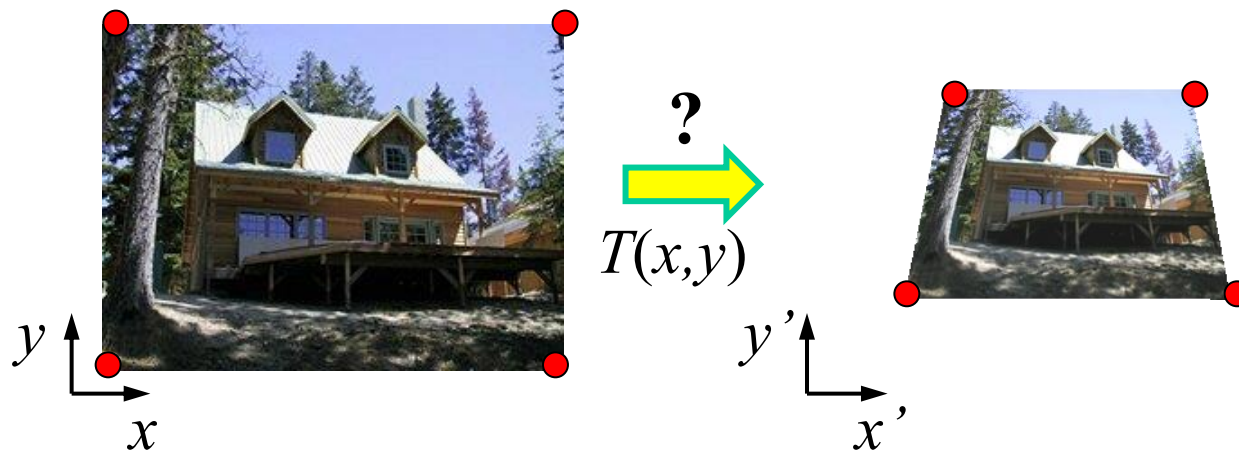


How many correspondences needed for affine?

How many DOF?

# Projective: # correspondences?

---

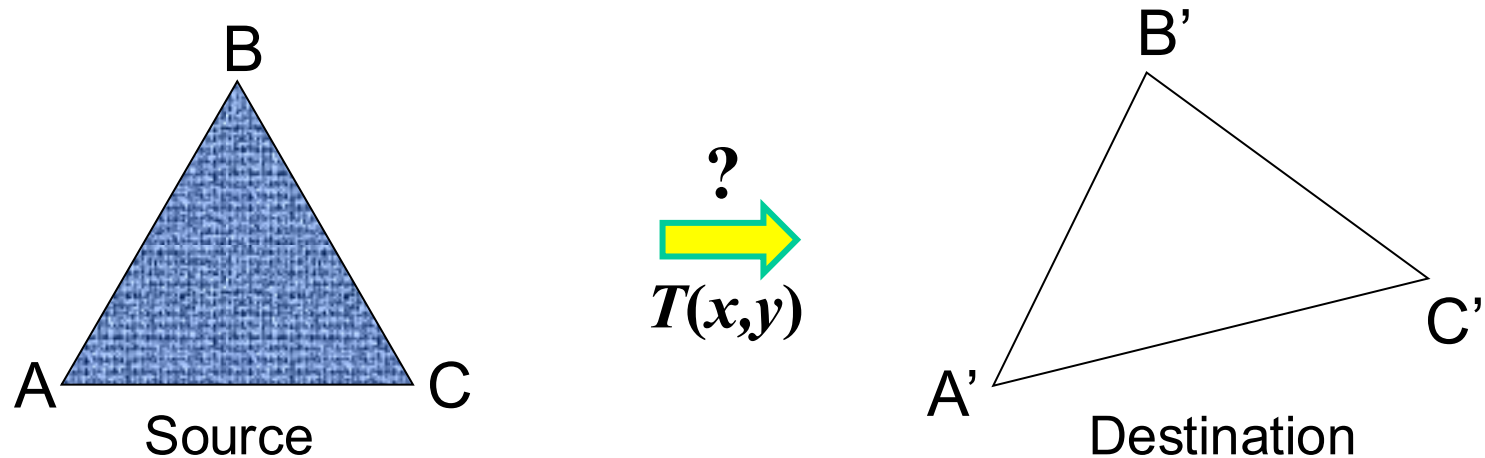


How many correspondences needed for projective?

How many DOF?

# Example: warping triangles

---



Given two triangles: ABC and A'B'C' in 2D (12 numbers)  
Need to find transform T to transfer all pixels from one to the other.

What kind of transformation is T?

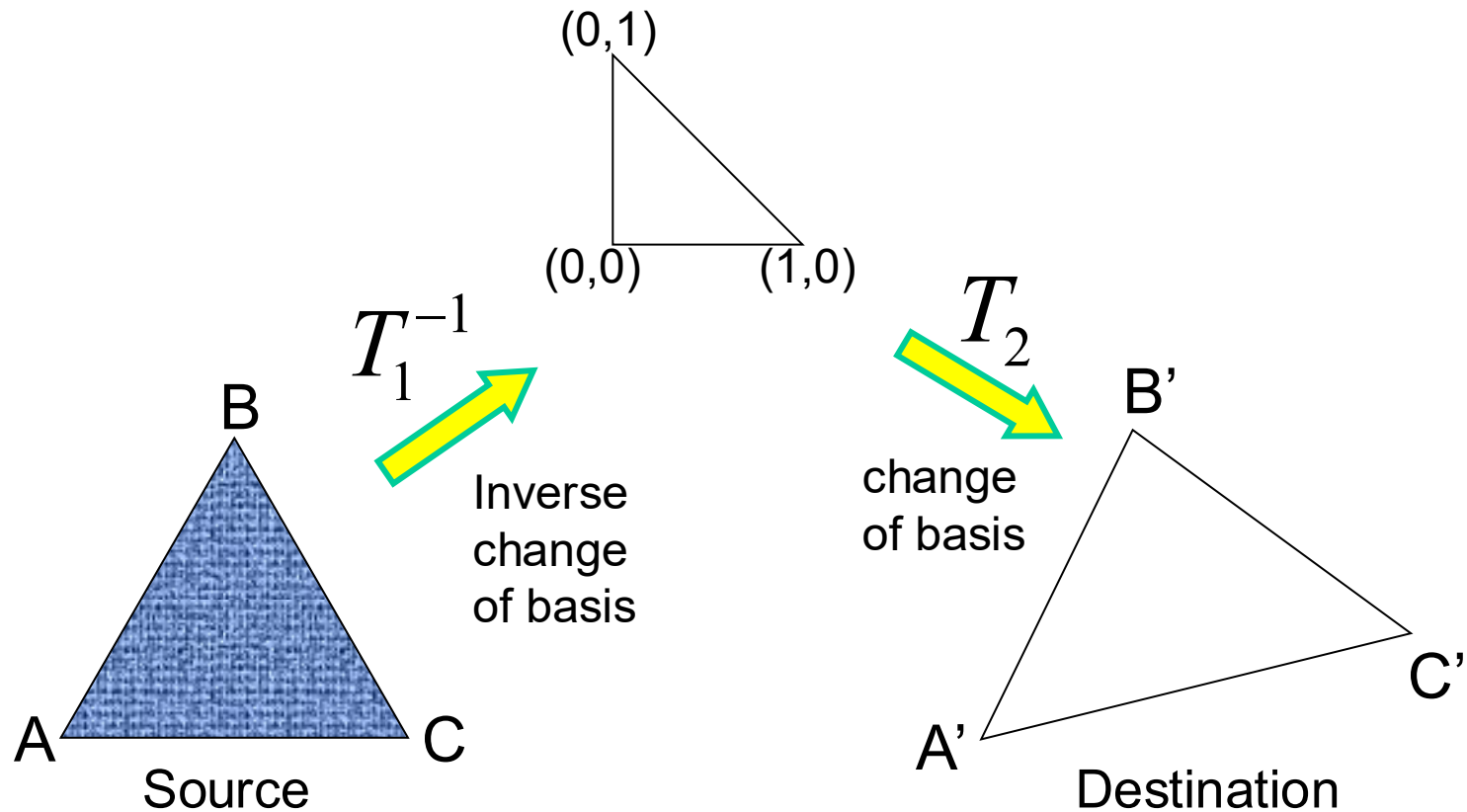
How can we compute the transformation matrix:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Two ways:  
Algebraic and  
geometric

# warping triangles (Barycentric Coordinates)

---



Don't forget to move the origin too!

# Morphing = Object Averaging

---



The aim is to find “an average” between two objects

- Not an average of two images of objects...
- ...but an image of the average object!
- How can we make a smooth transition in time?
  - Do a “weighted average” over time  $t$

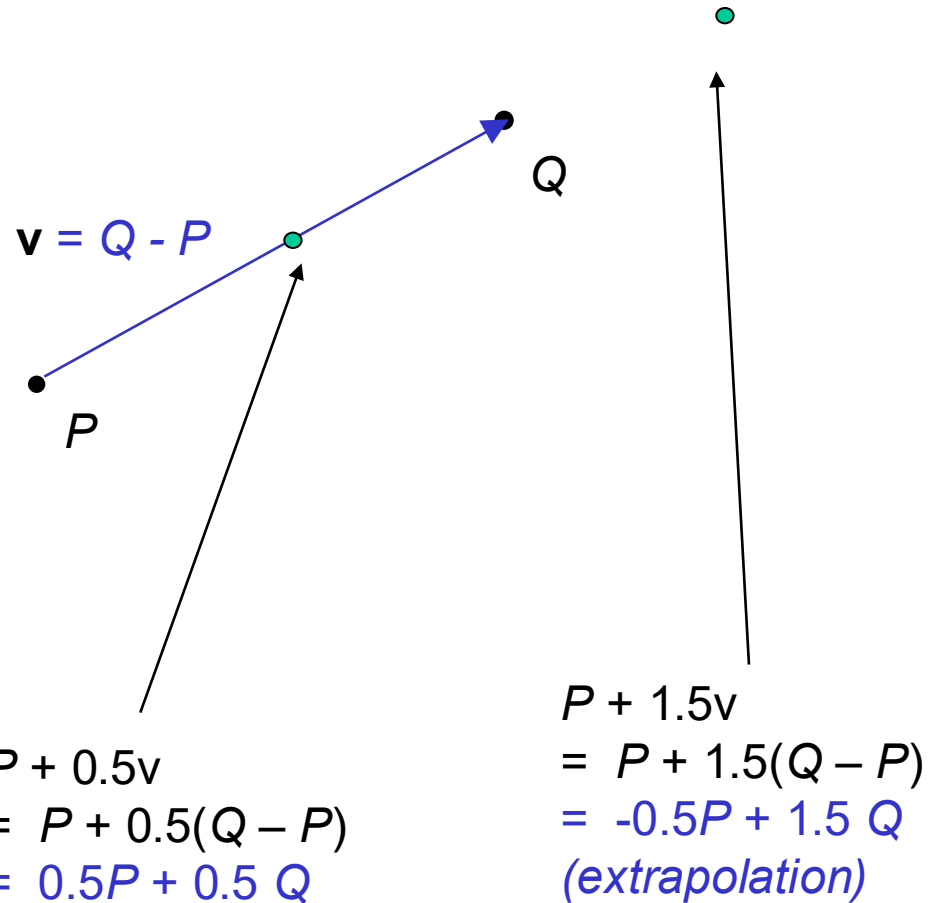
How do we know what the average object looks like?

- We haven't a clue!
- But we can often fake something reasonable
  - Usually required user/artist input

# Averaging Points

---

What's the average  
of P and Q?



Linear Interpolation  
(Affine Combination):  
New point  $aP + bQ$ ,  
defined only when  $a+b = 1$   
So  $aP+bQ = aP+(1-a)Q$

P and Q can be anything:

- points on a plane (2D) or in space (3D)
- Colors in RGB or HSV (3D)
- Whole images (m-by-n D)... etc.

# Idea #1: Cross-Dissolve

---



Interpolate whole images:

$$\text{Image}_{\text{halfway}} = (1-t) \cdot \text{Image}_1 + t \cdot \text{Image}_2$$

This is called **cross-dissolve** in film industry

But what if the images are not aligned?

# Idea #1: Cross-Dissolve

---



Cross-dissolve with images that are not aligned

# Idea #2: Align, then cross-dissolve

---



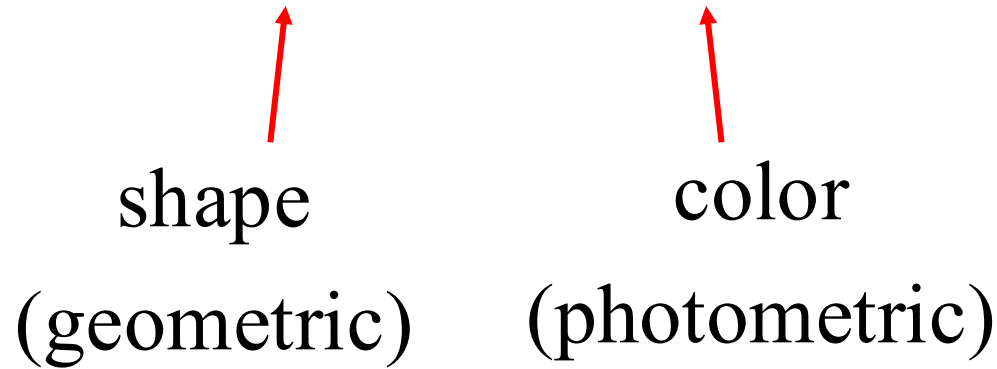
Align first, then cross-dissolve

- Alignment using global warp – picture still valid

# Image Morphing

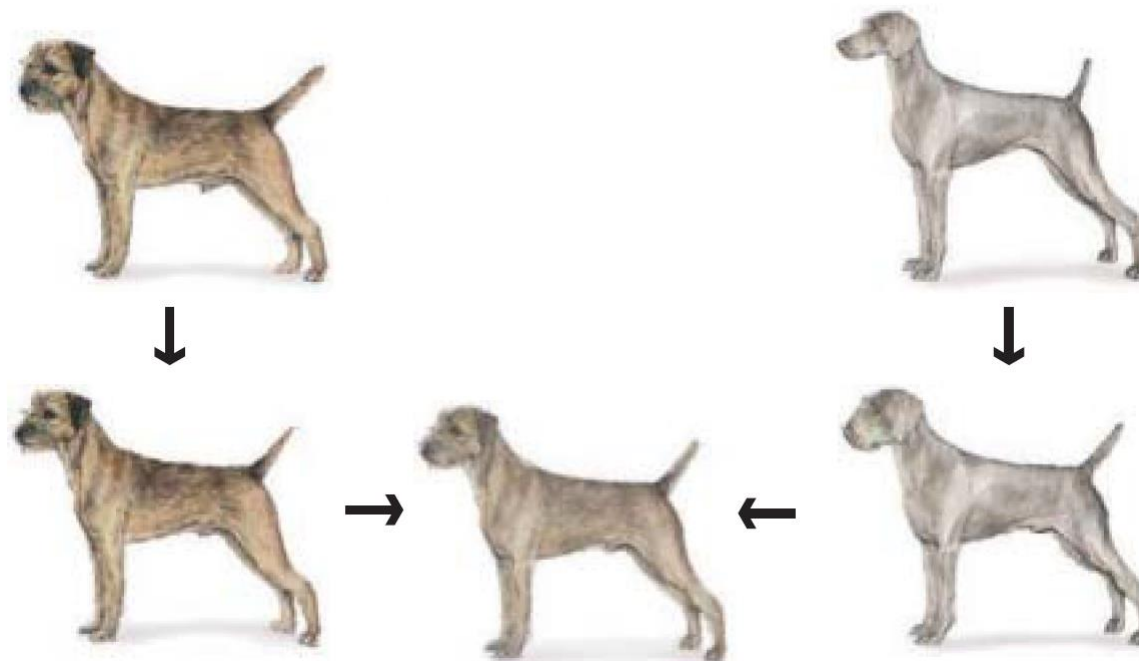
---

Morphing = warping + cross-dissolving



# Two-stage Morphing Procedure

---



Morphing procedure:

*for every  $t$ ,*

1. Find the average shape (the “mean dog” 😊)
  - warping
2. Find the average color
  - Cross-dissolve the warped images

# BUT: global warp not always enough!

---



What to do?

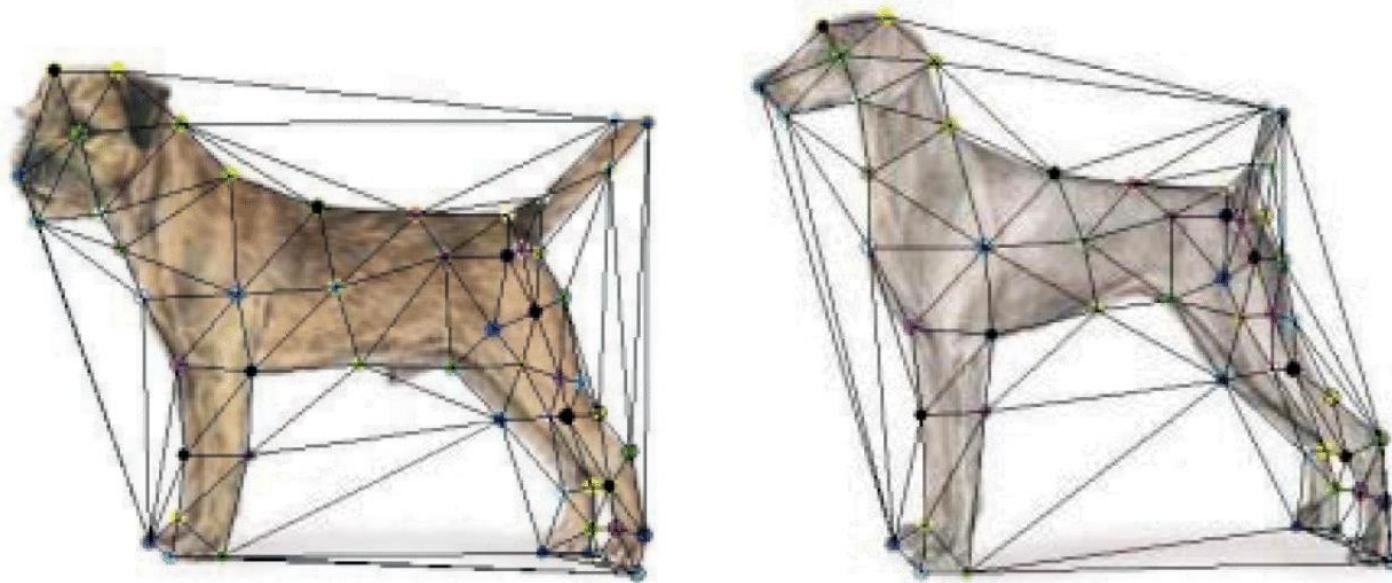
- Cross-dissolve doesn't work
- Global alignment doesn't work
  - Cannot be done with a global transformation (e.g. affine)
- Any ideas?

Feature matching!

- Nose to nose, tail to tail, etc.
- But what to do with all the intermediate pixels?

# Triangular Mesh

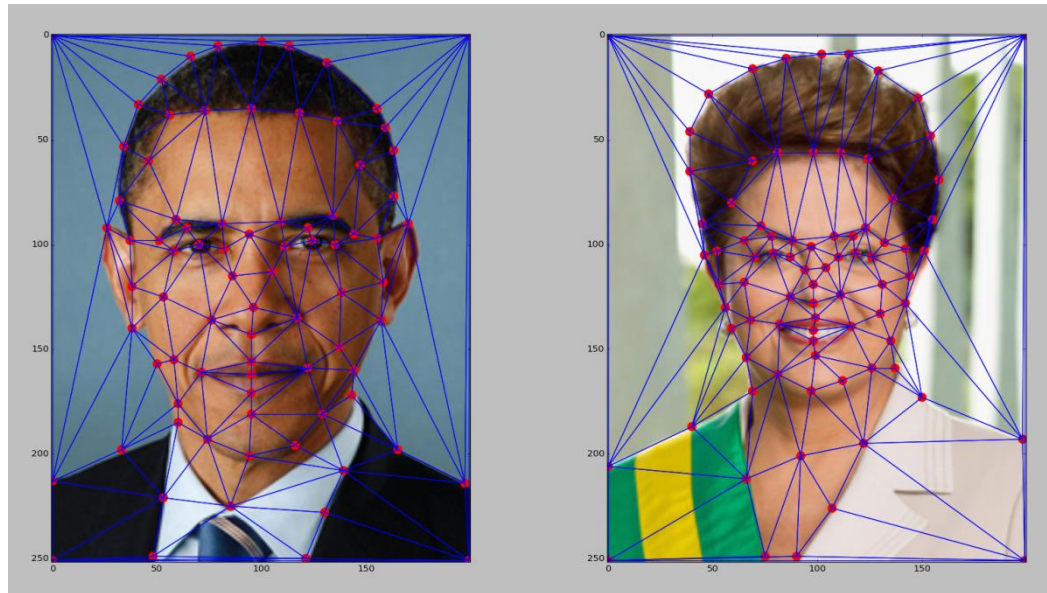
---



1. Input correspondences at key feature points
2. Define a triangular mesh over the points
  - Same mesh in both images!
  - Now we have triangle-to-triangle correspondences
3. Warp each triangle separately from source to destination
  - How do we warp a triangle?

# Full morphing result

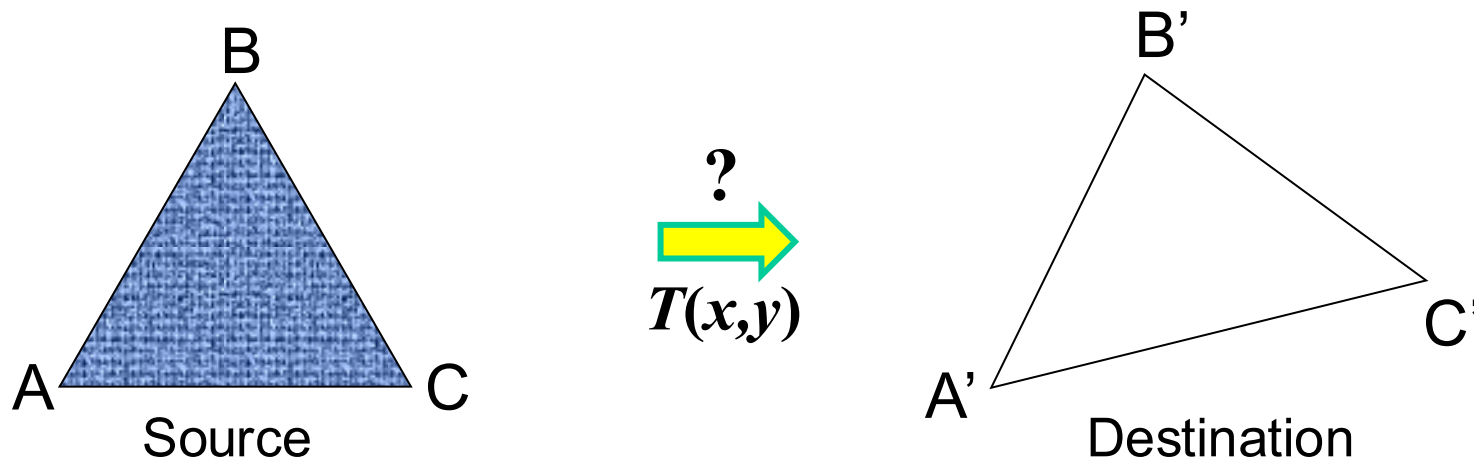
---



(c) Ian Albuquerque Raymundo da Silva

# Warping triangles

---



Given two triangles: ABC and A'B'C' in 2D (12 numbers)  
Need to find transform T to transfer all pixels from one to the other.

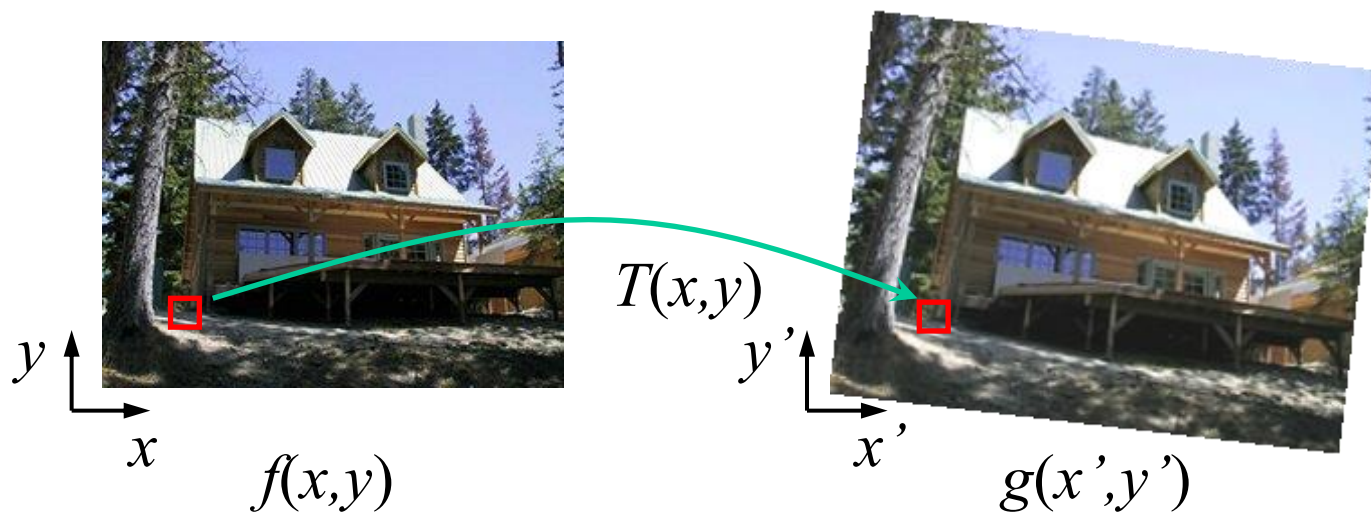
What kind of transformation is T?

How can we compute the transformation matrix:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

# Warping Pixels

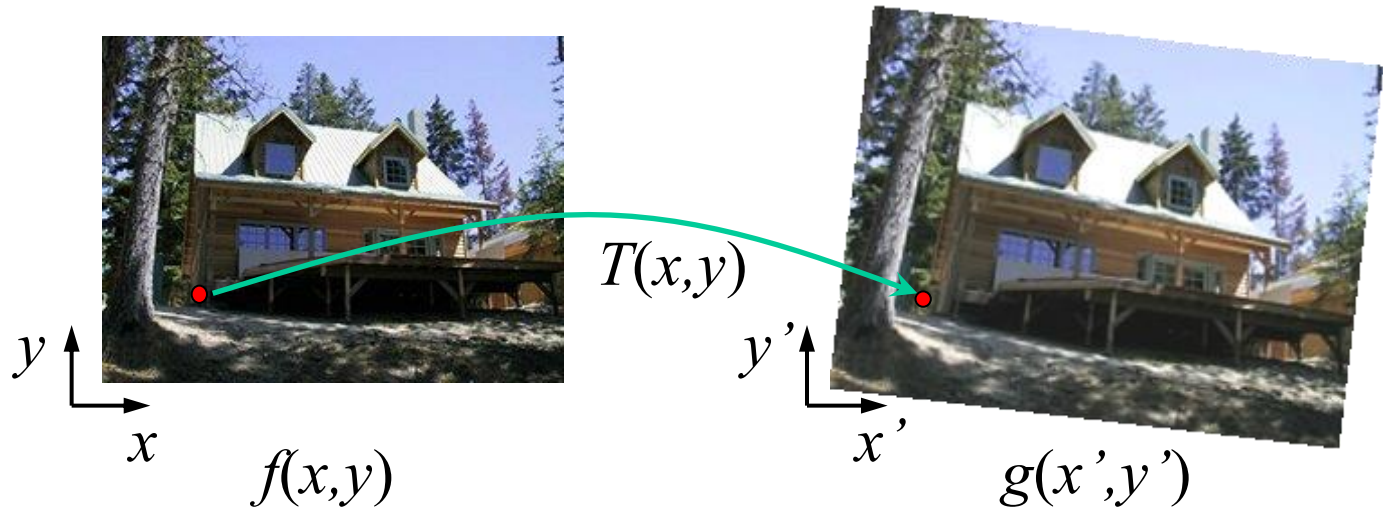
---



Given a coordinate transform  $(x',y') = T(x,y)$  and a source image  $f(x,y)$ , how do we compute a transformed image  $g(x',y') = f(T(x,y))$ ?

# Forward warping

---

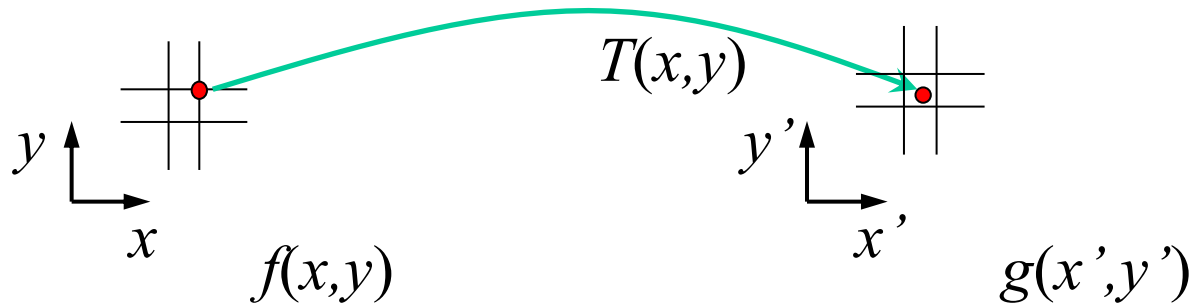


Send each pixel  $f(x,y)$  to its corresponding location  
 $(x',y') = T(x,y)$  in the second image

Q: When does this work well, when not? If something goes wrong, what is it? Ideas to fix?

# Forward warping

---



Send each pixel  $f(x, y)$  to its corresponding location  
 $(x', y') = T(x, y)$  in the second image

Q: what if pixel lands “between” two pixels?

A: distribute color among neighboring pixels  $(x', y')$   
– Known as “splatting”

# Forward warping

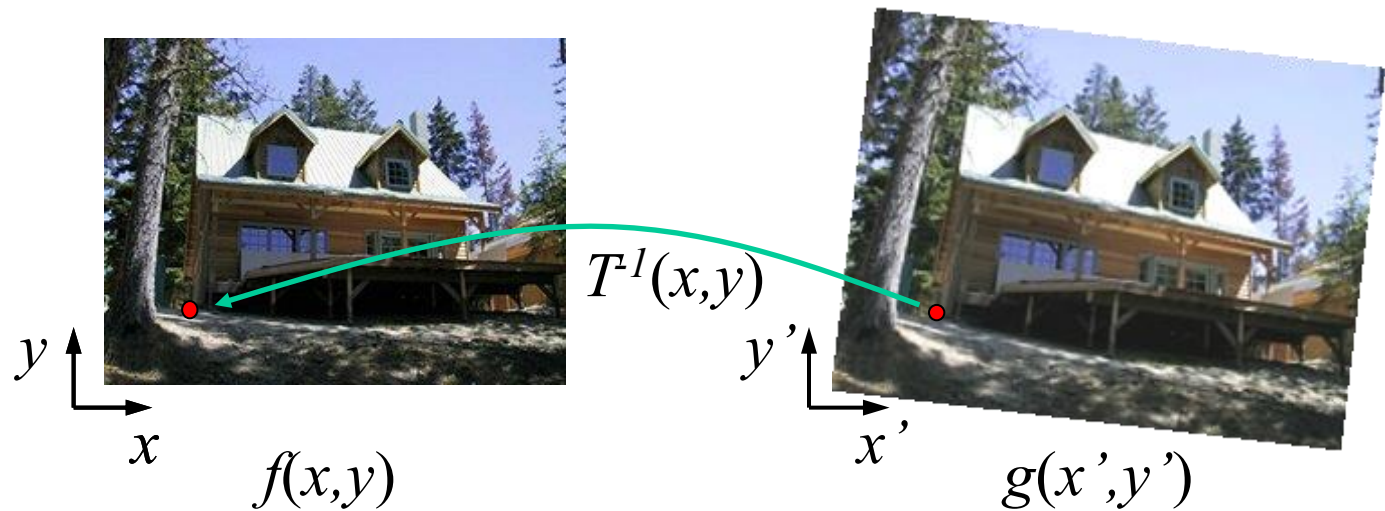
---



What would happen here?

# Inverse warping

---

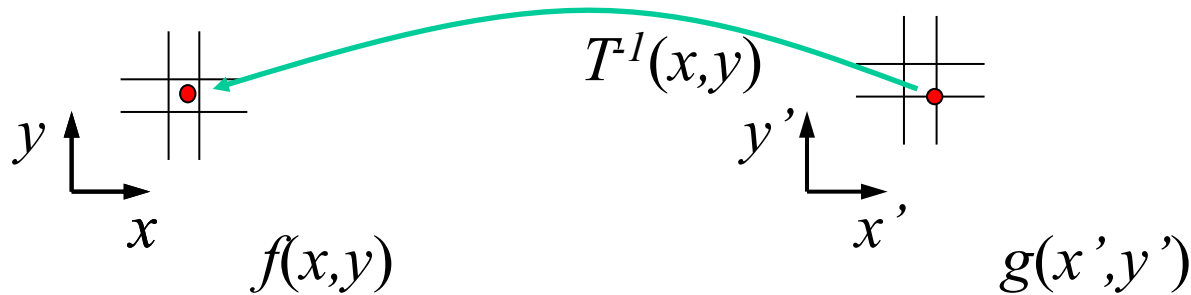


Get each pixel  $g(x',y')$  from its corresponding location  
 $(x,y) = T^{-1}(x',y')$  in the first image

Q: what if pixel comes from “between” two pixels?

# Inverse warping

---



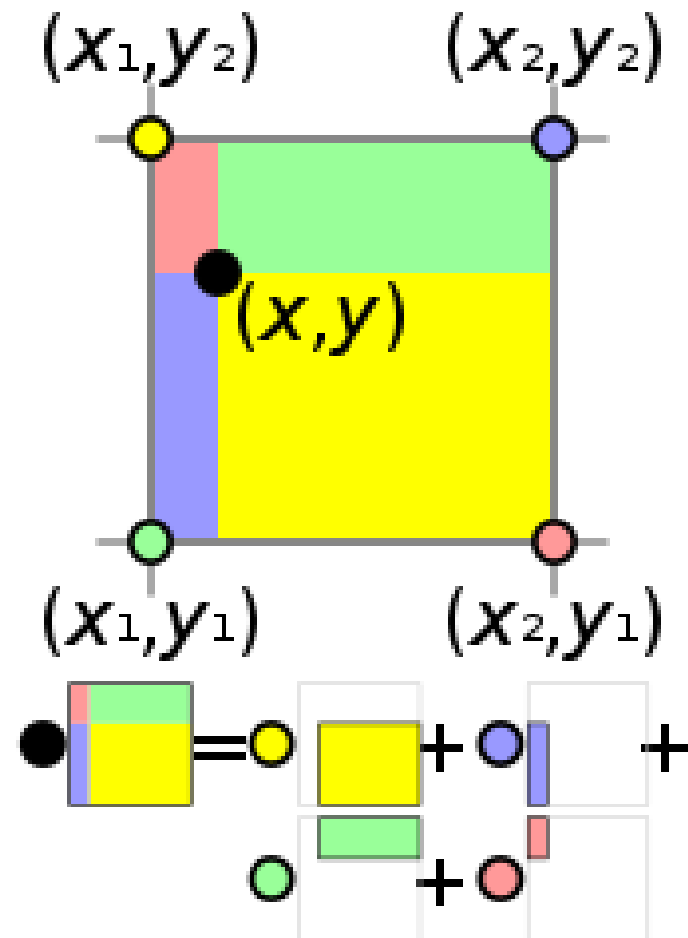
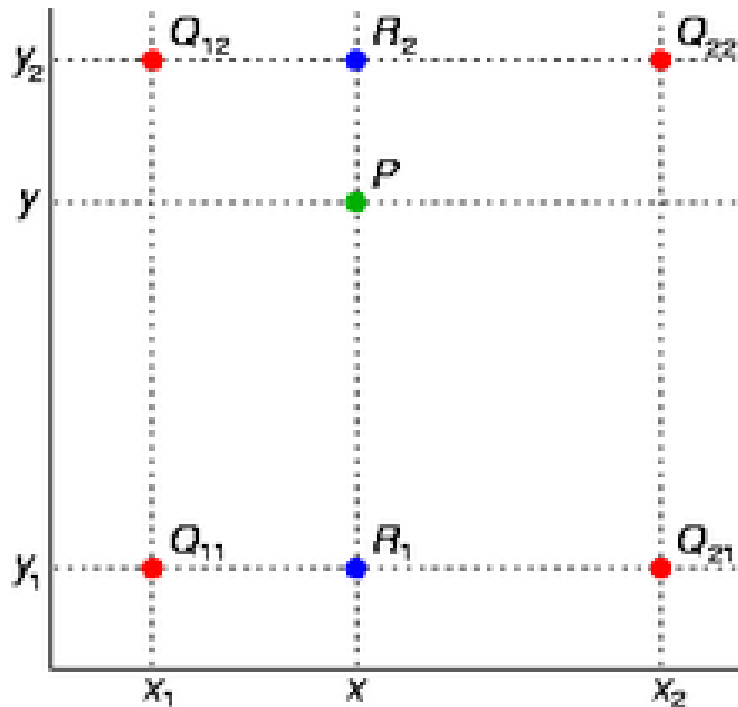
Get each pixel  $g(x', y')$  from its corresponding location  $(x, y) = T^{-1}(x', y')$  in the first image

Q: what if pixel comes from “between” two pixels?

A: *Interpolate* color value from neighbors

- nearest neighbor, bilinear, Gaussian, bicubic
- Check out `interp2` in Matlab / `remap` Python

# Bilinear Interpolation



[http://en.wikipedia.org/wiki/Bilinear\\_interpolation](http://en.wikipedia.org/wiki/Bilinear_interpolation)  
 Help interp2

# Forward vs. inverse warping

---

Q: which is better?

A: usually inverse—eliminates holes

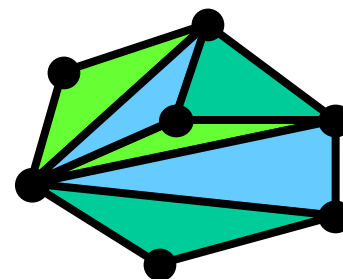
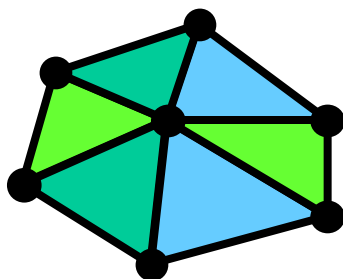
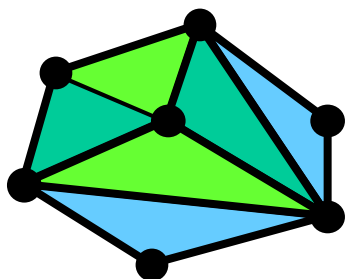
- however, it requires an invertible warp function—not always possible...

# Triangulations

---

A *triangulation* of set of points in the plane is a *partition* of the convex hull to triangles whose vertices are the points, and do not contain other points.

There are an exponential number of triangulations of a point set.

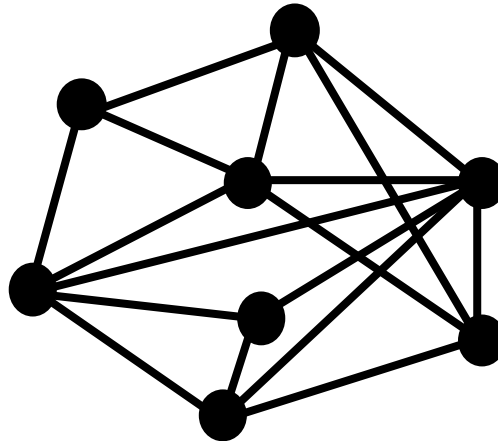


# An $O(n^3)$ Triangulation Algorithm

---

Repeat until impossible:

- Select two sites.
- If the edge connecting them does not intersect previous edges, keep it.



# “Quality” Triangulations

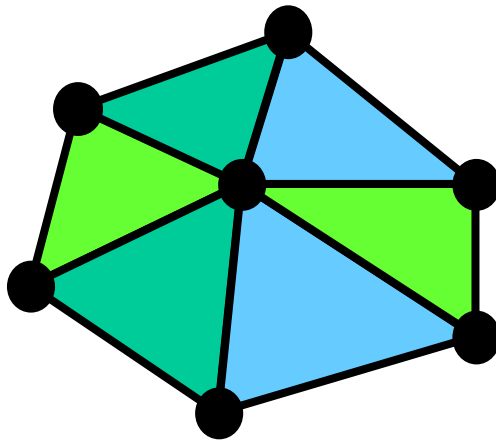
---

Let  $\alpha(T) = (\alpha_1, \alpha_2, \dots, \alpha_{3t})$  be the vector of angles in the triangulation  $T$  in increasing order.

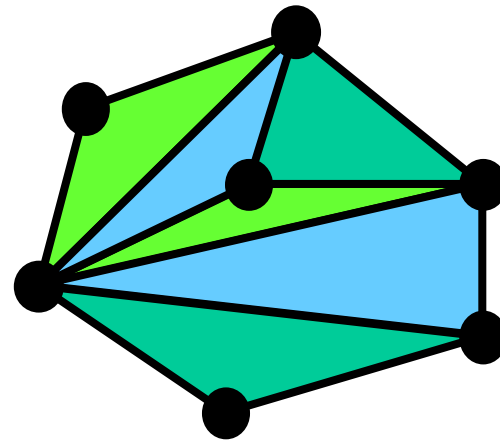
A triangulation  $T_1$  will be “better” than  $T_2$  if  $\alpha(T_1) > \alpha(T_2)$  lexicographically.

The Delaunay triangulation is the “best”

- Maximizes smallest angles



good

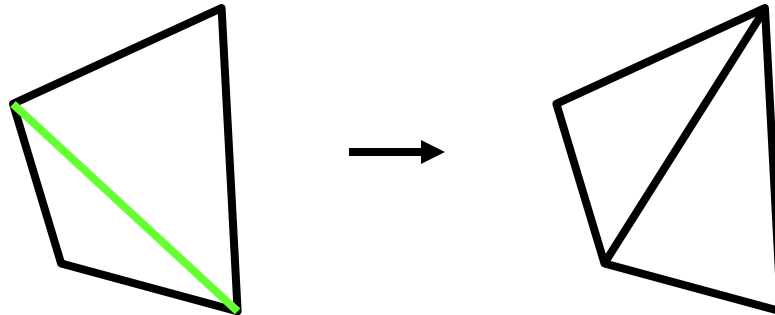


bad

# Improving a Triangulation

---

In any convex quadrangle, an *edge flip* is possible. If this flip *improves* the triangulation locally, it also improves the global triangulation.



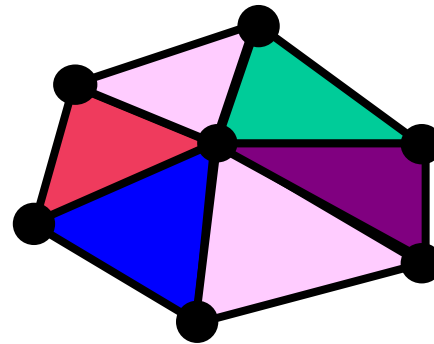
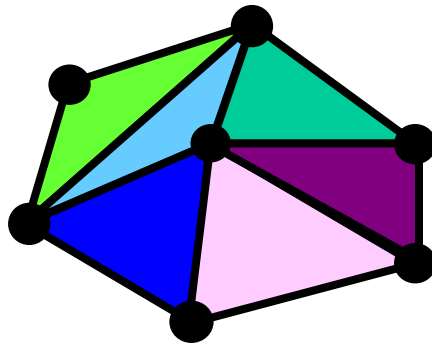
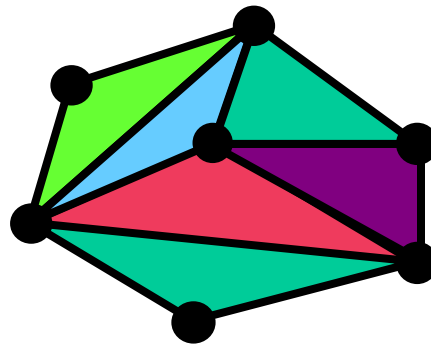
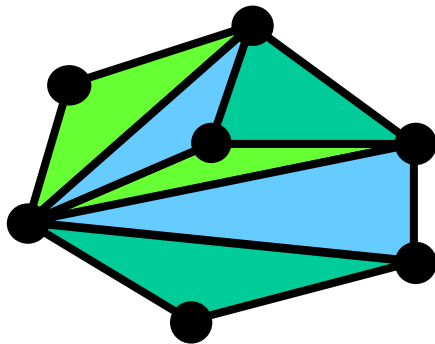
If an edge flip improves the triangulation, the first edge is called *illegal*.

# Naïve Delaunay Algorithm

---

Start with an arbitrary triangulation. Flip any illegal edge until no more exist.

Could take a long time to terminate.



# Delaunay Triangulation by Duality

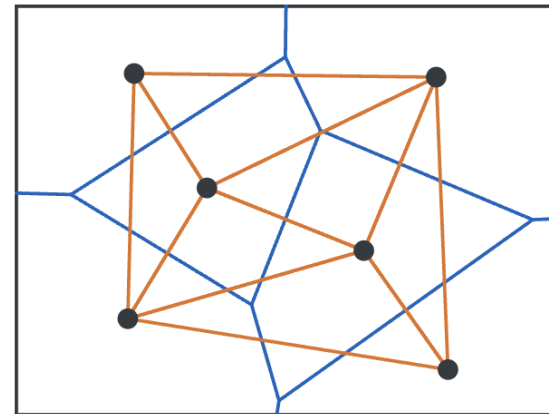
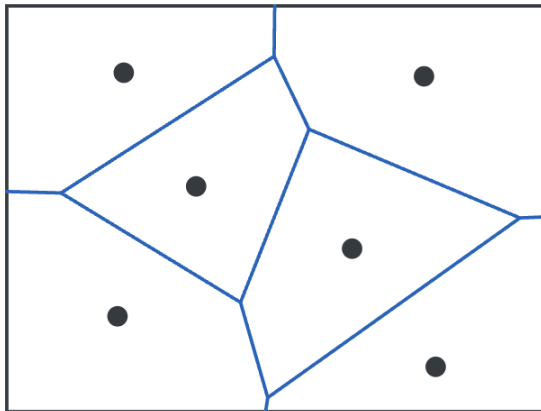
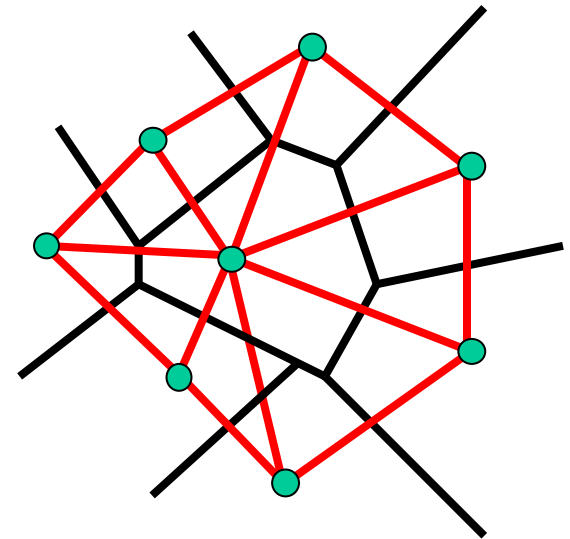
---

General position assumption: There are no four co-circular points.

Draw the dual to the Voronoi diagram by connecting each two neighboring sites in the Voronoi diagram.

**Corollary:** The DT may be constructed in  $O(n \log n)$  time.

This is what Matlab's `delaunay` function uses.

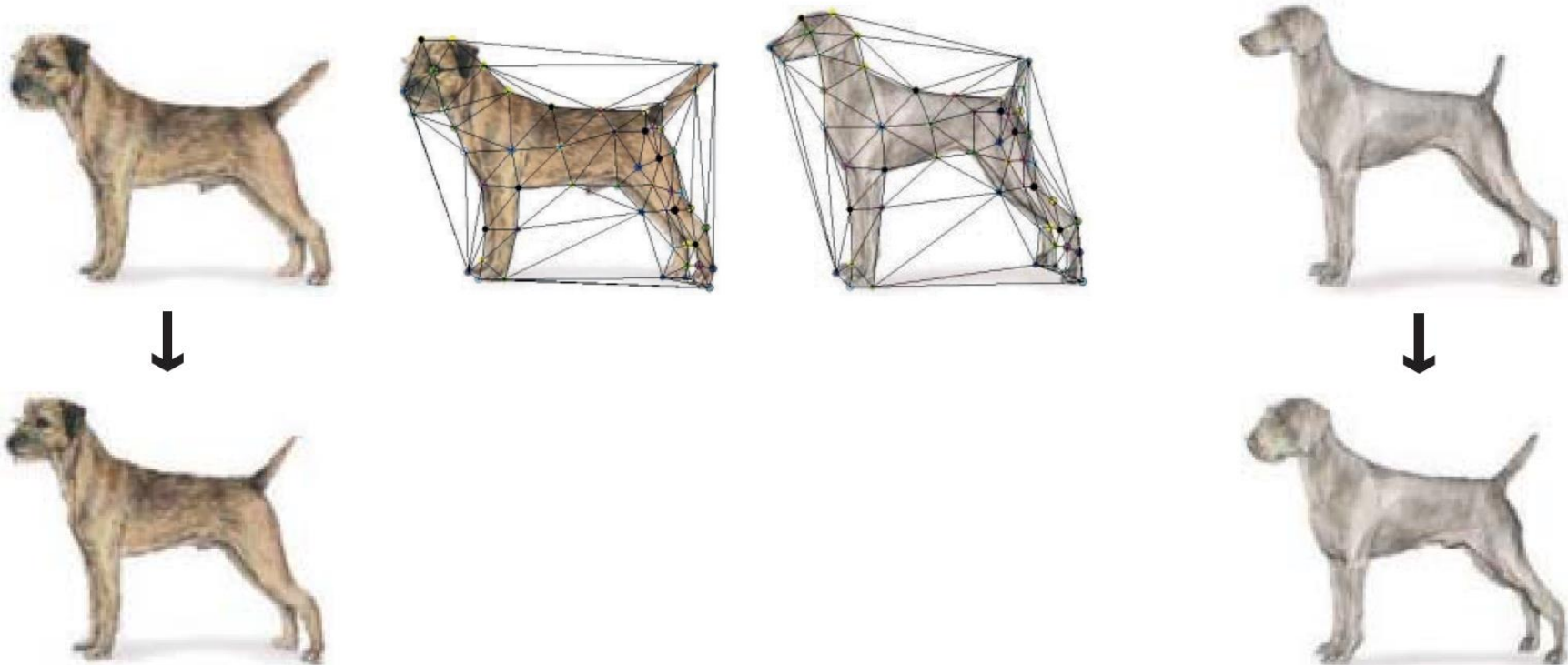


# 1. Create Average Shape

---

How do we create an intermediate warp at time  $t$ ?

- Assume  $t$  is in  $[0,1]$
- Simple linear interpolation of each feature pair  
 $p=(x,y) \rightarrow p'(x,y)$
- $(1-t)*p+t*p'$  for corresponding features  $p$  and  $p'$



## 2. Create Average Color

---



Interpolate whole images:

$$\text{Image}_{\text{halfway}} = (1-t) * \text{Image} + t * \text{image}'$$

**cross-dissolve!**



# Morphing

---

1. Define corresponding points
  2. Define triangulation on points
    - Use same triangulation for both images
  3. For each  $t = 0:\text{step}:1$ 
    - a. Compute the average shape at  $t$  (weighted average of points)
    - b. For each triangle in the average shape
      - Get the affine projection to the corresponding triangles in each image
      - For each pixel in the triangle, find the corresponding points in each image and set value to weighted average (cross-dissolve each triangle)
    - c. Save the image as the next frame of the sequence
- Life-hack: can be done with just two nested loops (for  $t$ , and for each triangle). Hint: compute warps for all pixels first, then use `interp2`

Extras

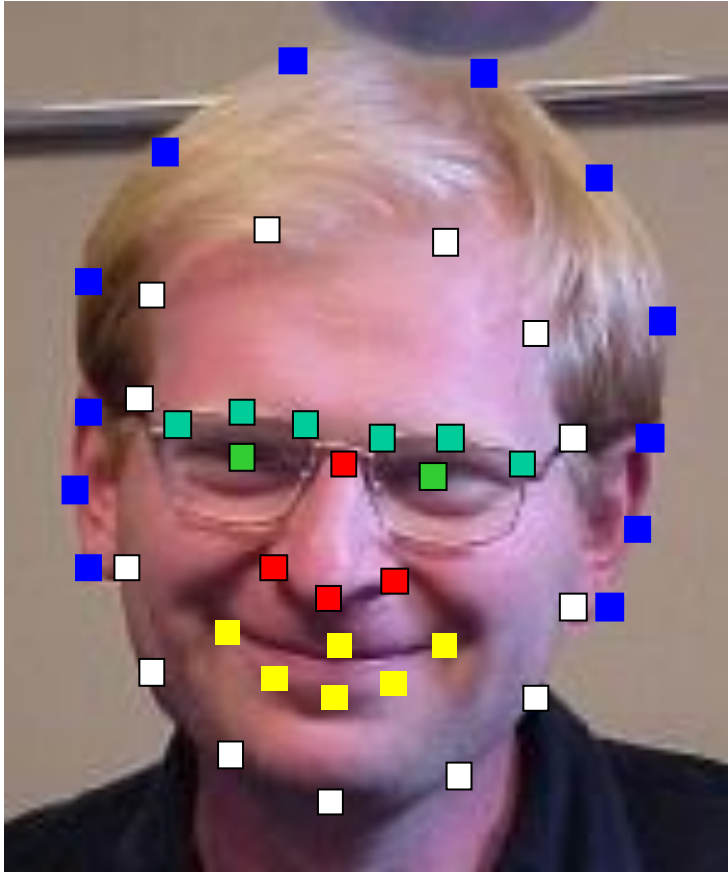
# How to align faces?

---



# Shape Vector

---



Provides alignment!

=



# Appearance Vectors vs. Shape Vectors

Appearance  
Vector



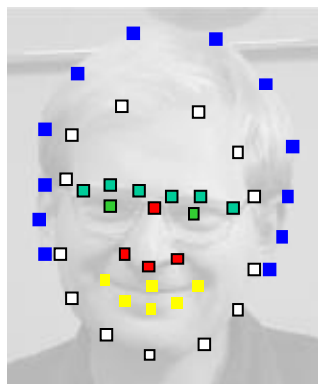
200\*150 pixels (RGB)



Vector of  
 $200*150*3$   
Dimensions

- Requires Annotation
- Provides alignment!

Shape  
Vector



43 coordinates (x,y)



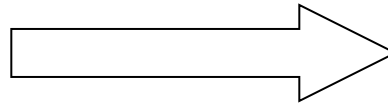
Vector of  
 $43*2$   
Dimensions

# Average Face

---



1. Warp to mean shape
2. Average pixels



# Subpopulation means

---

Examples:

- Male vs. female
- Happy vs. sad
- Angry Kids
- People wearing glasses
- Etc.
- <http://www.faceresearch.org>



Average female



Average kid



Average happy male



Average male

# Average Women of the world

---



Central African

Burmese

Cambodian

English

Ethiopian

Filipino



Greek

Indian

Iranian

Irish

Israeli

Italian



Peruvian

Polish

Romanian

Russian

Samoan

South African

# Average Men of the world

---



AUSTRIA



AFGHANISTAN



ARGENTINA



BURMA (MYANMAR)



GERMANY



GREECE



CAMBODIA



ENGLAND



ETHIOPIA



FRANCE



IRAQ



IRELAND



MONGOLIA



PERU



POLAND



PUERTO RICO



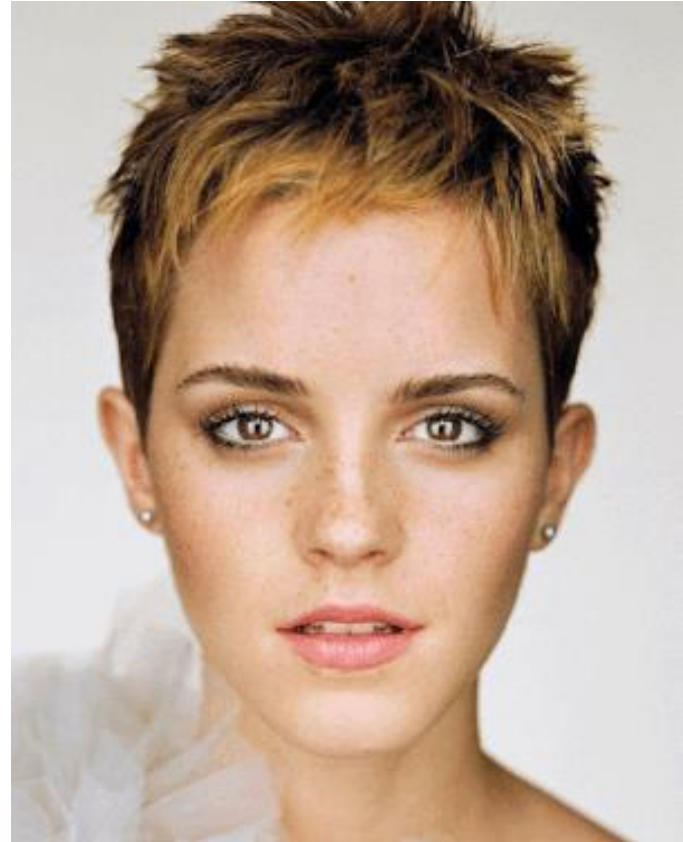
UZBEKISTAN



AFRICAN AMERICAN

# Examples

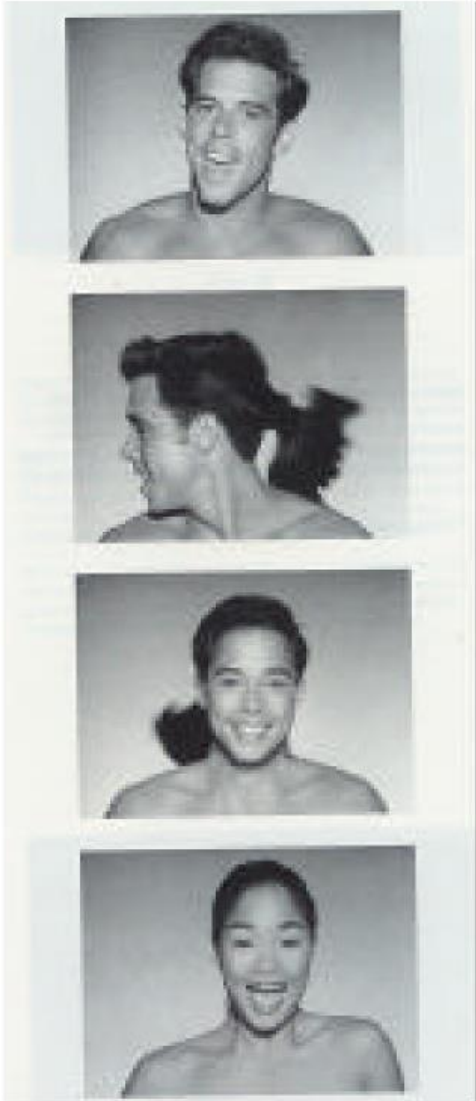
---



© Rachel Albert, CS194-26, Fall 2015

# Dynamic Scene (“Black or White”, MJ)

---



<https://vimeo.com/456846170>